Task 1

Naive Approach

Time Complexity: H x W x K^2

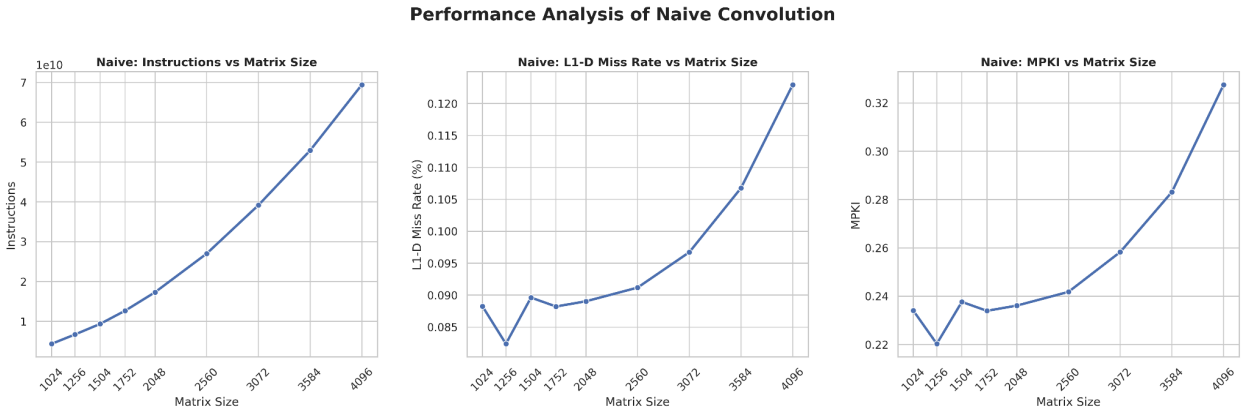

Figure 1.1

Instructions vs Matrix Size:
The graph depicts a quadratic growth between matrix size and the number of instructions.

L1-D Miss Rate and MPKI vs Matrix Size:
As we are not using any optimization techniques, as the number of instructions grows the L1-D miss rate increases parallely. That's why we can see the quadratic growth pattern in both the graphs against matrix sizes.

Task 1A

1. What motivated you to make changes to your code?

Reordering
Restructuring the nested loops to ky, kx, oy, ox attempts to improve spatial locality in memory. By placing the image dimensions (oy, ox) innermost, the CPU reads and writes to consecutive memory addresses in the in and out arrays, theoretically maximizing the use of fetched cache lines.

Unrolling
Expanding the inner loops manually removes the overhead of loop condition checks, pointer arithmetic, and branch mispredictions. By hardcoding standard filter sizes (K=3, K=5), kernel weights are fetched into registers exactly once per output pixel, exposing higher instruction-level parallelism to the hardware.

2. What considerations did you take into account when implementing those changes?

Reordering:
Accumulating values across different kernel iterations means a local scalar variable (acc) can no longer hold the intermediate sum for a single pixel. The algorithm must explicitly initialize the output array to zero during the first pass (if(ky == 0 && kx == 0)) and subsequently use += to accumulate values directly into main memory.

Unrolling:

Explicit unrolling introduces significant code complexity to prevent out-of-bounds memory accesses. While fixed K sizes get clean, dedicated paths, generic K values require calculating a safe unroll boundary (kend = K - (K % 3)). A secondary cleanup loop must then be maintained to process leftover iterations that do not cleanly divide by the unroll factor.

3. How effective are your changes? (Which metric will you use to quantify effectiveness and why?)

Reordering

This change severely degrades performance on realistic workloads. According to Figure 2.1 the speedup falls below the baseline to 0.52x on larger matrices. The most vital metric here is L1-D Cache MPKI (Misses Per Kilo-Instruction). The analysis shows L1-D MPKI skyrocketing from roughly 0.2 (naive) to nearly 16.0. This massive increase indicates severe cache thrashing, completely overriding any benefit from a slightly lower overall instruction count.

Unrolling:

This optimization is highly successful. The Figure 2.2 confirms a consistent 1.26x to 1.31x speedup across all matrix sizes. The defining metrics are Speedup and reduced L1-D Miss Rate / MPKI. Even though the raw instruction count increases due to duplicated code, the significantly more efficient cache access patterns and pipeline utilization yield a definitively faster execution.

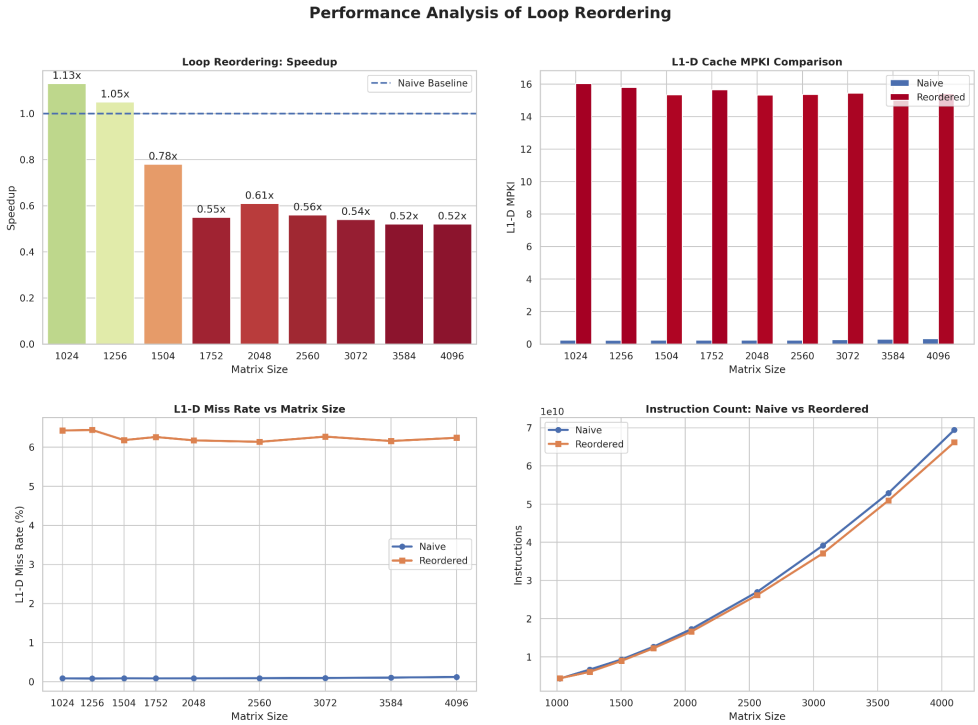

Figure 2.1

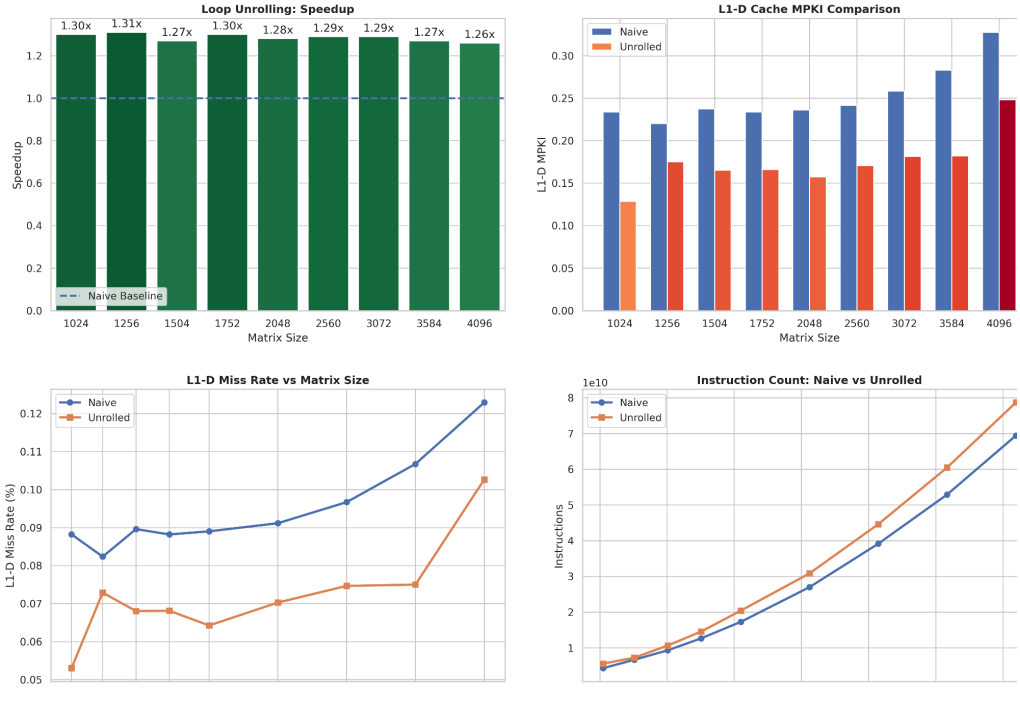

Figure 2.2

TASK 1B

1. Report the changes in L1-D MPKI observed when moving from naive to tiled 2D convolution. Justify your observations.

Observation: Implementing cache tiling successfully reduces the L1-D Misses Per Kilo-Instruction (MPKI) for optimal tile sizes, while severely worsening it for suboptimal ones. For example, a tile size of 64 maintains a low, stable L1-D Miss rate (between 0.078 and 0.094) across various matrix sizes.

Justification: The naive implementation iterates across the entire width of the image. For large matrices, by the time the loop returns to the next row of the kernel, the previously accessed input data has likely been evicted from the L1 cache. Tiling restructures the traversal into localized blocks (e.g., 64x64). This ensures that the subset of the image and output currently being processed remains resident in the L1 cache, maximizing temporal locality.

2. How did L1-D MPKI vary across different matrix sizes and tile sizes? Explain your findings in terms of the cache hierarchy and working set sizes.

Variation: Small tile sizes (like 8) yielded the worst performance, with MPKI scaling drastically higher (reaching roughly 0.35) as the matrix size increased. Conversely, tile sizes of 32, 64, and 100 maintained a much flatter, lower MPKI curve (clustering around 0.20 to 0.25) regardless of how large the matrix grew.

Explanation: This behavior is governed by the L1 cache working set size. A tile size of 8 is too small; it fails to fully utilize the spatial locality of an entire cache line (usually 64 bytes) and incurs heavy overhead constantly jumping between tiny blocks. A tile size of 64 hits the "sweet spot" where the tile's memory footprint fits comfortably inside the L1-D cache capacity. Once the tile fits, the cache absorbs the repeated memory accesses,

preventing the L1-D miss rate from inflating even when the global matrix size expands to 4096.

3. Did you achieve a speedup? If yes, quantify the improvement and identify the contributing factors. If not, analyze the limiting factors and propose possible solutions.

Improvement Quantified: A consistent speedup was achieved for all tile sizes strictly greater than 8. The highest average speedup reached 1.20x for a tile size of 64, peaking at 1.27x for a 1024x1024 matrix.

Contributing Factors: The primary driver for this 1.20x speedup is the significant reduction in L1-D Cache miss rates, which fell to as low as 0.078 for optimal tiles. By restricting the inner processing loops to bounded tile dimensions, the CPU spends less time stalled waiting for main memory fetches.

Limiting Factors (Slowdowns): Tiling with a size of 8 resulted in a performance degradation, yielding an average speedup of only 0.82x. This happens because the algorithmic overhead of managing the extra nested loops (oy, ox, ty, tx) and branch instructions outweighs the memory access benefits when the tile is too small to meaningfully improve cache reuse.

Figure 3.1

Task 1C

1. Report the change in the number of instructions you observed when moving from the naive to the SIMD implementation. Justify your observations.

Observation: The total number of executed instructions decreases significantly, roughly by a factor of 8 for AVX2 (256-bit) and 4 for SSE (128-bit) during the core calculation.

Justification: In the naive version, the inner loops process one pixel at a time. In the SIMD version, the ox loop increments by 8 (ox += 8) or 4 (ox += 4). A single SIMD instruction replaces multiple scalar arithmetic instructions, drastically reducing the total number of loop iterations and branch checks required to process the image.

2. Did you achieve any speedup? If so, how much, and what contributed to it? If not, what were the reasons?

The primary reason for the observed performance improvement is data-level parallelism.

In the naive implementation, each floating-point value is processed individually. Each convolution operation requires separate scalar instructions for loading, multiplication, and accumulation.

The SIMD implementation processes multiple floating-point values simultaneously.

For the 128-bit implementation, 4 single-precision floating-point values are processed per vector operation.

For the 256-bit implementation, 8 single-precision floating-point values are processed per vector operation.

The major factors contributing to the speedup are:

1. **Data-level parallelism.**
2. **Higher floating-point throughput.**
3. **Reduced loop overhead.**
4. **Reduced branch overhead.**
5. **Reduced indexing overhead.**
6. **Lower instruction count**

3. Which SIMD intrinsics did you use? Justify your choice of functions.

AVX2 256-bit Intrinsics:

```
__m256 acc = _mm256_setzero_ps();
```

This initializes the accumulator vector with eight floating-point zeros. Each vector lane accumulates the convolution result for a different output element.

```
__m256 input = _mm256_loadu_ps(row + kx);
```

This loads eight consecutive single-precision floating-point values into a 256-bit vector register.

The unaligned version was selected because the memory address may not always be aligned to a 32-byte boundary. Using _mm256_loadu_ps( ) allows the implementation to safely load data without requiring explicit memory alignment.

```
__m256 weight = _mm256_set1_ps(ker[ky * K + kx]);
```

This broadcasts a single kernel value to all eight vector lanes.

```
_mm256_mul_ps(input, weight);
```

This performs eight single-precision floating-point multiplications simultaneously.

```
_mm256_add_ps(acc, product);
```

This performs eight floating-point additions simultaneously and accumulates the convolution results.

```
_mm256_storeu_ps(&out[oy * W + ox], acc);
```

This stores the eight computed output values to memory.

The unaligned store is used because the output address may not always be aligned to a 32-byte boundary

The 128-bit implementation uses equivalent SSE intrinsics:

- _mm_setzero_ps()
- _mm_loadu_ps()
- _mm_set1_ps()
- _mm_mul_ps()
- _mm_add_ps()
- _mm_storeu_ps()

These perform the same operations as their AVX2 counterparts but operate on four single-precision floating-point values instead of eight.

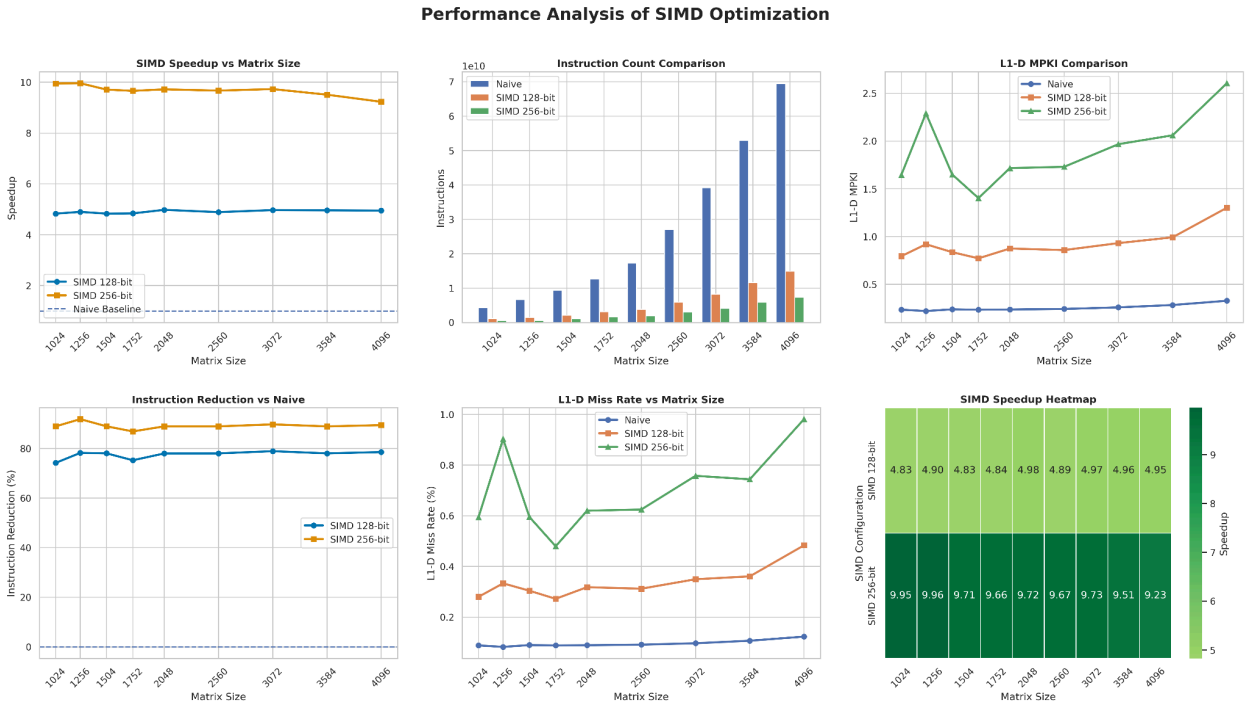

Figure 4.1

Task 1D

The objective of this part of the assignment was to investigate whether combining multiple optimization techniques can provide greater performance improvements than applying optimization techniques individually.

Implementation	Optimized Combination
Implementation 1	Reorder + Unroll
Implementation 2	Tiling + Unroll
Implementation 3	SIMD + Tiling
Implementation 4	Reorder + Tiling
Implementation 5	SIMD + Unroll

The performance was evaluated across:

- Matrix sizes: 1024 to 4096
- Kernel sizes: 3, 5, and 7
- Metrics:
    - Speedup based on matrix sizes and kernel sizes
    - Instruction count
    - L1-D MPKI

1. Instruction Count

- **Observations:** As shown in Figure 5.3, **SIMD + Unrolling** and **Tiling + SIMD** execute the lowest number of total instructions across all matrix sizes. Conversely, **Reorder + Tiling** yields the highest instruction count.
- **Justification:** SIMD instructions process 8 single-precision floating-point values concurrently using 256-bit AVX2 registers. Combining SIMD with loop unrolling eliminates outer loop control instructions and pointer arithmetic. In contrast, loop reordering requires repeatedly reading and writing intermediate values to memory (`+=` operations), which bloats the instruction count and forces redundant array indexing.

2. Cache Efficiency (L1-D MPKI)

- **Observations**: Reorder + Unroll suffers from a massive L1-D MPKI spike (~9.8 MPKI) that remains consistently high across all matrix sizes. Non-reordered methods (SIMD + Unrolling, Tiling + SIMD, Tiling + Unroll) keep L1-D MPKI extremely low (~0.8 to 1.0 MPKI).
- **Justification**: Placing the spatial loops (`oy, ox`) innermost during loop reordering forces the inner loop to continuously sweep across large image rows for each single kernel weight. This constantly evicts active cache lines. By keeping spatial

locality contiguous, cache hits remain high and cache line thrashing is avoided.

3. Speedup Across Implementations

- **Observations**:
  - SIMD + Unrolling achieves the best overall performance, peaking at a 11.95x speedup for K=5 and maintaining 7.18x to 7.65x speedups for K=3 and K=7 (Figure 5.2). Across matrix sizes, it stays consistently well above the naive baseline (Figure 5.1).
  - Tiling + SIMD is the second-best strategy, reaching up to 7.04x speedup for K=7.
  - Reorder + Unroll and Reorder + Tiling frequently perform worse than the naive baseline (down to 0.52x to 0.54x speedup for large matrices).
- **Justification**:
  - Why SIMD + Unrolling Wins: Combining explicit K=3 and K=5 loop unrolling with AVX2 fused multiply-accumulate (_mm256_fmadd_ps) allows the compiler to broadcast kernel weights into vector registers once. The hardware computes 8 output pixels concurrently per cycle without the indexing and conditional branching overhead introduced by cache tiling loops.

- Why Reordering Fails: The cache locality penalties of loop reordering outweigh any register reuse benefits. The CPU pipeline spends significant time stalled waiting on memory bus fetches due to the high L1-D MPKI.

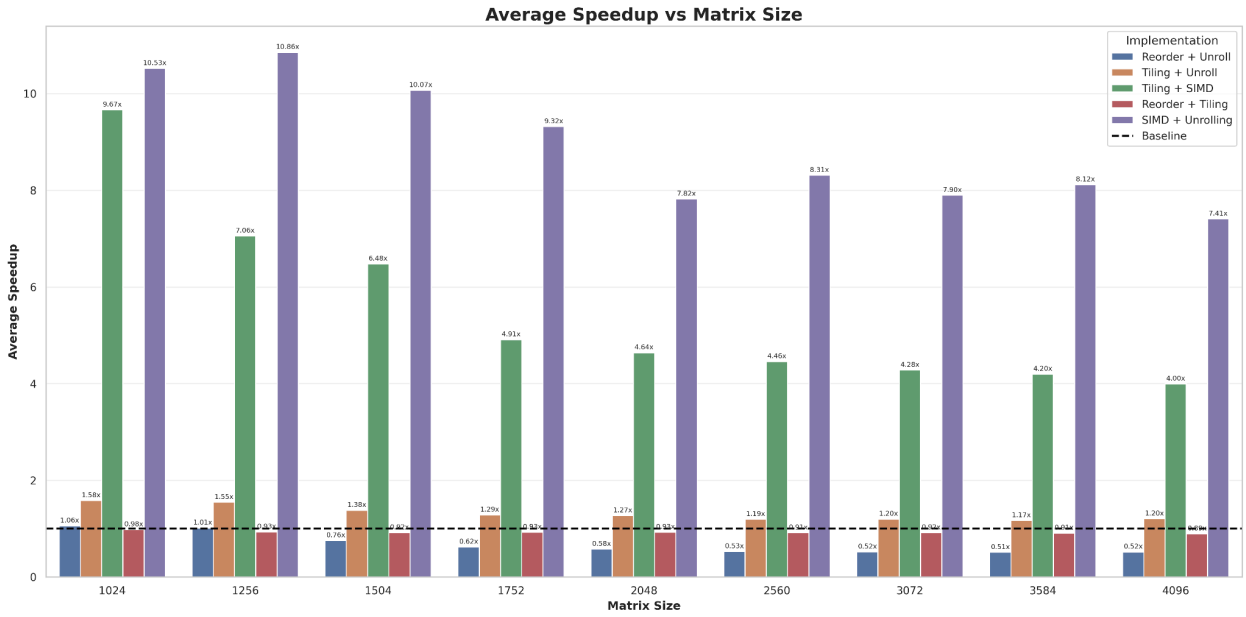

Figure 5.1

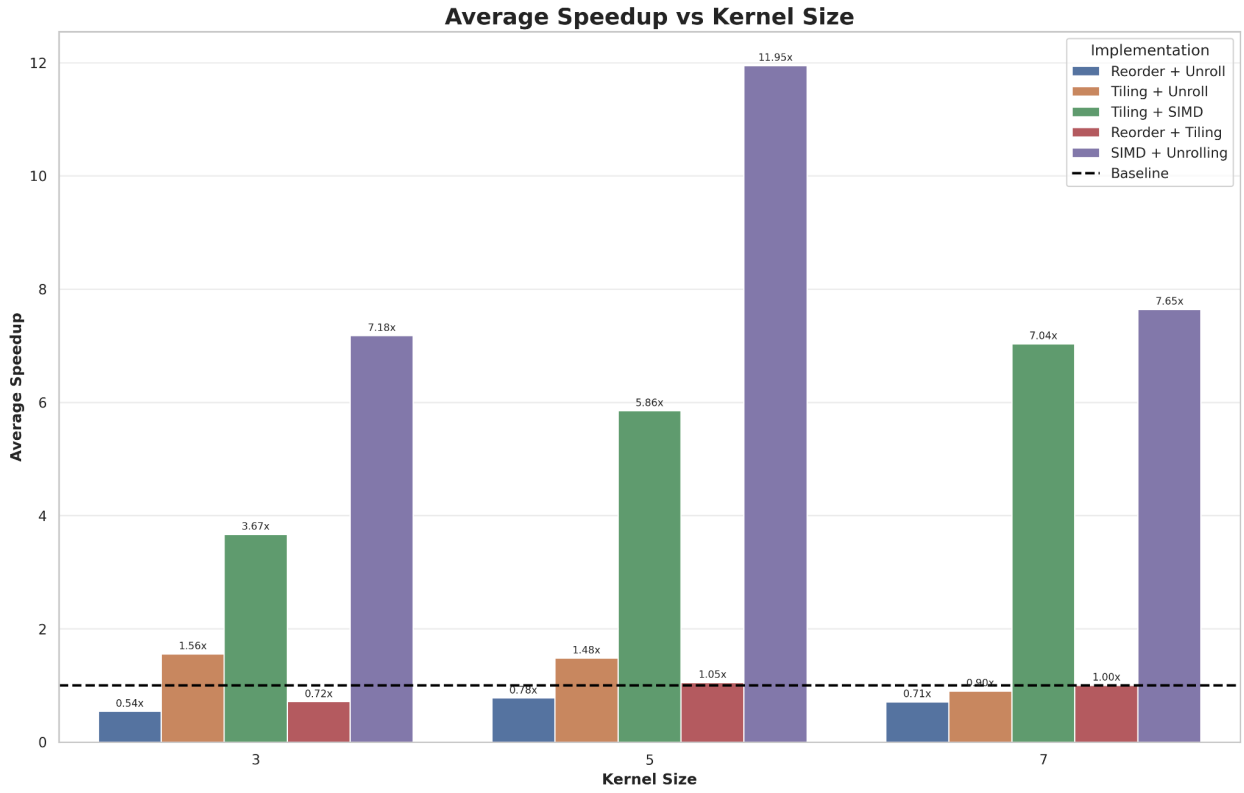

Figure 5.2

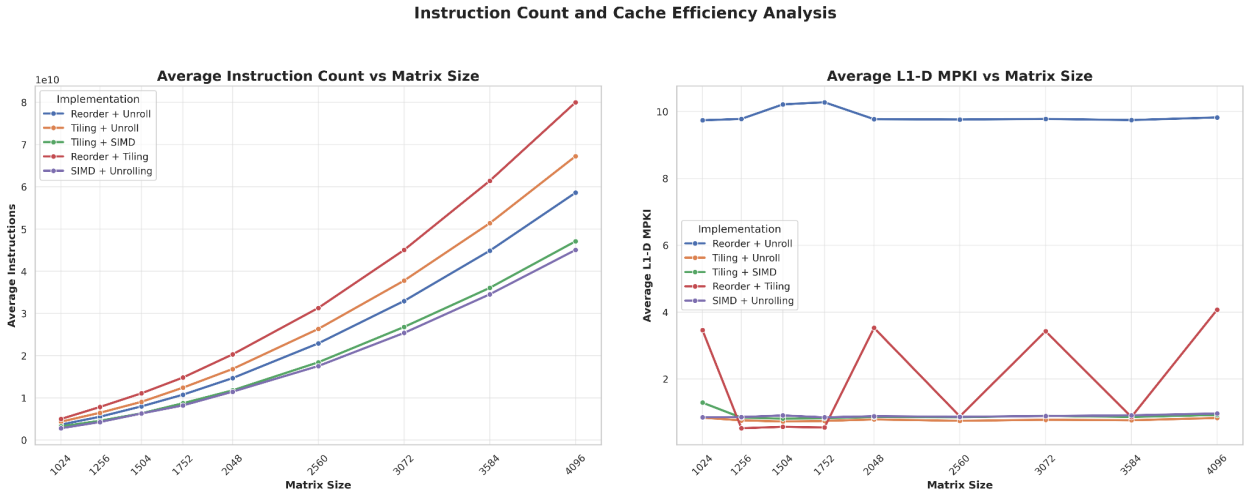

Figure 5.3